



RELATÓRIO TÉCNICO

DataChat SQL Lite

*Um assistente conversacional Text-to-SQL sobre o Brazilian E-Commerce Public Dataset
(Olist)*

Djhonatah Wesley, Mirelle Casimiro e Filipe Antonny

Campina Grande — PB

2026

SUMÁRIO

1. Introdução	3
2. Concepção e Estudo de Domínio	4
2.1 Estudo da Base Olist	4
Visão Geral	4
2.2 Importação dos Arquivos CSV	4
2.3 Definição da Arquitetura da Solução	5
3. Integração de Dados e Text-to-SQL	7
3.1 Integração com o Banco de Dados	7
3.2 Implementação do Módulo Text-to-SQL	7
3.3 Desenvolvimento do Backend	8
3.4 Primeiros Testes com Perguntas Simples	8
4. Orquestração e Estabilidade	9
4.1 Integração Completa com LLM	9
4.2 Tratamento de Erros e Estabilidade	9
4.3 Implementação do Histórico de Consultas	9
4.4 Validação com Consultas Multi-tabela	10
5. Interface e Consolidação Final	11
5.1 Desenvolvimento da Interface	11
5.2 Explicações em Linguagem Natural e Formatação	11
5.3 Testes Finais e Validação	12
5.4 Documentação, Vídeo e Apresentação	12
6. Conclusão	13

1. Introdução

Este relatório técnico documenta o desenvolvimento do projeto DataChat SQL Lite, uma aplicação conversacional que permite consultar, em linguagem natural, uma base de dados relacional, com tradução automática das perguntas para consultas SQL (Text-to-SQL). O domínio de dados utilizado é o Brazilian E-Commerce Public Dataset (Olist), amplamente utilizado em projetos de análise de comércio eletrônico.

O trabalho foi conduzido ao longo de quatro semanas, com entregas incrementais: concepção e estudo do domínio; integração de dados e implementação do núcleo Text-to-SQL; orquestração, tratamento de erros e testes de estabilidade; e, por fim, o desenvolvimento da interface e a consolidação final da aplicação. As seções seguintes apresentam o detalhamento técnico de cada uma dessas etapas.

2. Concepção e Estudo de Domínio

2.1 Estudo da Base Olist

Visão Geral

A base de dados Olist contém informações reais e anonimizadas de e-commerce brasileiro.

- **Objetivo da Base:** analisar o comércio eletrônico no Brasil, compreendendo vendas, pagamentos, logística, produtos e avaliações de clientes.
- **Período dos Dados:** pedidos realizados entre 2016 e 2018.
- **Volume:** aproximadamente 100 mil pedidos.
- **Tabelas:** 8 tabelas relacionais utilizadas no projeto.

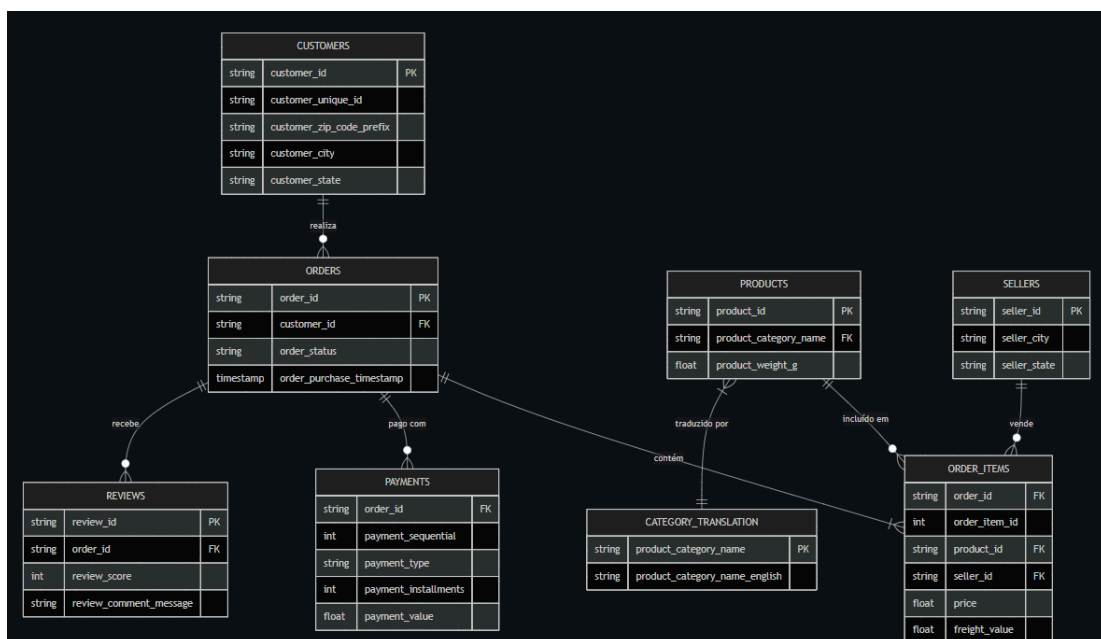


Figura 1 — Diagrama Entidade-Relacionamento da base Olist.

2.2 Importação dos Arquivos CSV

Os arquivos foram importados para um banco de dados DuckDB. O papel de cada arquivo é detalhado a seguir:

Arquivo	Tabela	PK	FKs Principais	Descrição
olist_orders_dataset.csv	orders	order_id	customer_id	Detalhes do pedido (status, datas).
olist_order_items_dataset.csv	order_items	Composta	order_id, product_id	Itens, preços e fretes.
olist_products_dataset.csv	products	product_id	N/A	Físico e categoria.

2.3 Definição da Arquitetura da Solução

O sistema adotará uma arquitetura modular focada na integração eficiente de um Large Language Model (LLM) com o banco de dados.

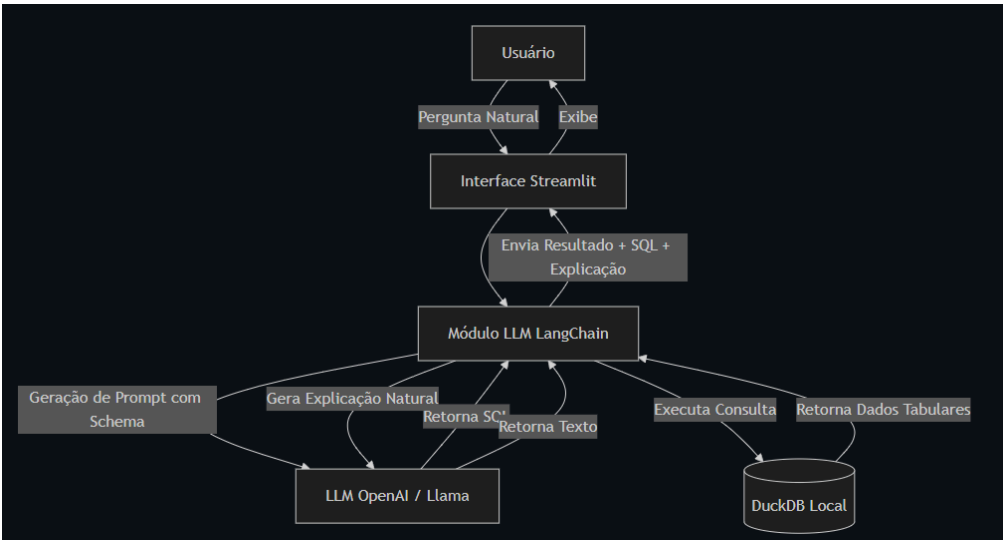


Figura 2 — Arquitetura da solução DataChat SQL Lite.

Componentes:

1. **Frontend:** interface construída em Streamlit para fácil interação, com histórico de consultas e placeholders.
2. **Orquestrador LLM:** utilização do framework LangChain (com ferramentas de SQLDatabaseChain ou create_sql_agent) para injetar o DDL das tabelas no prompt, orientando o LLM.

3. **Database Engine:** DuckDB operando de forma serverless e otimizada para queries analíticas (OLAP).
4. **LLM Engine:** API de modelo de linguagem responsável por traduzir perguntas para SQL (Text-to-SQL) e explicar os dados de retorno.

3. Integração de Dados e Text-to-SQL

3.1 Integração com o Banco de Dados

Foi desenvolvido o módulo `src/db.py`, responsável por realizar a integração direta com o DuckDB. O componente atua como a camada de dados e fornece:

- **Extração do Esquema:** uma função que extrai dinamicamente o DDL de todas as tabelas, incluindo amostras de dados, essencial para munir o LLM com o contexto necessário.
- **Conexão Segura:** uso de `read_only=True` ao instanciar conexões para evitar injeção e alterações não desejadas nos dados.
- **Execução:** um executor de queries simples que padroniza os retornos no formato tabular usando `pandas.DataFrame`.

3.2 Implementação do Módulo Text-to-SQL

A tradução da linguagem natural para SQL foi implementada no módulo `src/text_to_sql.py`:

- **Modelos e APIs:** optou-se pela utilização do LangChain para facilitar a criação e a estruturação das chains. Como LLM, foi escolhido o LLaMA 3.3 70B executado por meio da provedora Groq, garantindo que as respostas sejam altamente velozes e precisas.

Critério	LLaMA 3.3 70B	Gemini 1.5 Flash	GPT-4o
Velocidade	Altíssima	Alta	Média
Custo	Muito Baixo	Muito Baixo	Alto

Tabela 1 — Comparação entre os modelos.

- **Prompt Engineering:** foi desenvolvido um System Prompt detalhado, injetando as instruções e restrições obrigatórias (ex.: limitar linhas, tratar NULL, não enviar

markdown ou texto avulso, aderir ao DuckDB) juntamente com o DDL da estrutura do banco.

3.3 Desenvolvimento do Backend

O orquestrador do sistema, presente no arquivo `src/backend.py`, foi criado para alinhar todo o fluxo da requisição de forma sequencial:

1. Extração do schema atualizado (`db.py`).
2. Passagem da pergunta e do schema para o LLM gerar a SQL (`text_to_sql.py`).
3. Recuperação da query gerada e envio para o banco (`db.py`).
4. Retorno centralizado dos resultados em formato estruturado.

3.4 Primeiros Testes com Perguntas Simples

Para atestar o pleno funcionamento do núcleo, implementamos um script de testes (`src/test_text_to_sql.py`) contendo apenas perguntas de validação simples, de acordo com o escopo desta semana. Os testes avaliados e executados com sucesso foram:

- “*Quantos pedidos existem na base?*”
- “*Quantos clientes únicos realizaram compras?*”
- “*Qual categoria possui mais produtos?*”

Todos resultaram em geração SQL perfeitamente válida e dados precisos.

4. Orquestração e Estabilidade

4.1 Integração Completa com LLM

A integração completa da aplicação com a inteligência artificial foi finalizada, orquestrando todo o pipeline.

- **Implementação:** através do `src/text_to_sql.py` e `src/backend.py`.
- **Dinâmica:** o aplicativo capta a pergunta do usuário via interface e envia para o backend. O backend puxa o schema, injeta no prompt em conjunto com a pergunta e envia à API do Groq (llama-3.3-70b-versatile). O modelo devolve a query formatada que é executada contra o DuckDB, garantindo que o ciclo completo de text-to-SQL-to-data funcione nativamente.

4.2 Tratamento de Erros e Estabilidade

A aplicação foi protegida contra falhas comuns durante a inferência ou na execução das consultas.

- **Captura de Exceções:** blocos `try/except` foram implementados no `src/backend.py` e `src/text_to_sql.py`.
- **Tratamento no Frontend:** se o modelo gerar um SQL inválido (alucinação) ou ocorrer um erro de conexão/API, a exceção é convertida em um alerta visual amigável (`st.error()`) através do Streamlit no arquivo `app/app.py`, garantindo que o aplicativo nunca congele ou “quebre” bruscamente para o usuário.

4.3 Implementação do Histórico de Consultas

Foi adicionado um mecanismo de persistência de sessão para melhorar a experiência do usuário (UX).

- **Gerenciamento de Estado:** utilização intensiva do `st.session_state` no Streamlit (`app/app.py`).

- **Comportamento:** cada vez que o usuário executa uma consulta válida, os dados (pergunta original, query SQL executada e o DataFrame com o resultado) são salvos numa lista em memória.
- **Visualização:** o histórico é exposto numa aba dedicada na interface, renderizando expanders para que o usuário recupere rapidamente dados anteriores sem custo adicional de token e processamento.

4.4 Validação com Consultas Multi-tabela

Para testar o estresse e a aderência das queries num banco de dados normalizado e altamente relacional, desenvolvemos baterias de testes complexas no script `tests/test_multi_table_queries.py`.

- **Desafio Analítico:** o motor de SQL (DuckDB) precisou compilar e rodar queries que envolvem JOIN de 3 ou mais tabelas.

Cenários testados:

- Faturamento total por estado do cliente (envolvendo `orders`, `customers` e `order_items`).
- Top 5 vendedores por faturamento (envolvendo `sellers`, `order_items` e `orders`).
- Nota média de avaliação por categoria (envolvendo 5 tabelas simultâneas: `reviews`, `orders`, `order_items`, `products` e `category_translation`).
- Forma de pagamento mais usada por estado e pedidos entre estados diferentes.
- **Resultados:** todos os cenários rodaram com sucesso, indicando que a camada analítica está íntegra e robusta para cruzamentos complexos.

5. Interface e Consolidação Final

5.1 Desenvolvimento da Interface

A interface da aplicação foi integralmente desenvolvida utilizando o framework Streamlit, focando em uma experiência de usuário (UX) intuitiva e executiva.

- **Design Sóbrio e Moderno:** implementação de estilos CSS personalizados para ocultar elementos desnecessários da UI base, estilizar botões e áreas de chat, garantindo uma aparência profissional.
- **Consultas Recomendadas:** inclusão de botões rápidos (“Consultas Recomendadas”) para perguntas pré-definidas, facilitando o primeiro contato do usuário com a ferramenta.
- **Visualização em Abas:** resultados são separados em duas abas: “Visualização” (explicação natural e tabela de dados) e “SQL Gerada” (para auditoria da consulta elaborada pelo LLM).

5.2 Explicações em Linguagem Natural e Formatação

Para entregar valor de negócio e não apenas dados brutos, a aplicação gera uma explicação amigável para cada retorno:

- **Templates Dinâmicos:** o sistema analisa o DataFrame resultante e a pergunta do usuário para criar textos explicativos. Por exemplo, formatação de números em padrão brasileiro (R\$ e separador de milhar).
- **Fallback Genérico:** caso a consulta do LLM fuja dos padrões pré-mapeados, uma explicação genérica dinâmica é acionada, descrevendo o volume de registros e colunas retornadas.
- **Indicador de Fonte:** inserção de um marcador de fonte para que o usuário saiba exatamente de quais tabelas do banco DuckDB / Olist os dados foram extraídos.

5.3 Testes Finais e Validação

A aplicação passou por uma bateria de testes finais abordando todas as exigências obrigatórias do projeto.

- **Resiliência a Erros (RF08):** validado o comportamento da aplicação diante de perguntas fora de contexto (“OFF_TOPIC”) ou falhas de SQL, com a interface exibindo um erro de processamento não obstrutivo.
- **Consultas com JOIN (RF10):** verificado o funcionamento de consultas complexas envolvendo relacionamentos entre múltiplas tabelas simultaneamente (orders, order_items, products, customers, etc.).
- **Histórico (RF09):** o histórico funcional na barra lateral permite a revisão rápida e eficiente das interações prévias, evitando novos processamentos no LLM.

5.4 Documentação, Vídeo e Apresentação

Com o código finalizado, o esforço da Semana 4 consolidou-se nas entregas finais exigidas no edital:

- **Documentação Final:** consolidação de todos os relatórios semanais técnicos (contendo o cabeçalho acadêmico oficial da UEPB), além da finalização da configuração do projeto.
- **Gravação do Vídeo:** elaborado roteiro para o vídeo demonstrativo (até 5 minutos), focando em demonstrar o pipeline end-to-end: inserção da pergunta → geração LLM → SQL → DuckDB → resultado explicado.
- **Apresentação:** preparação do material para apresentação (10 a 15 minutos), enfatizando os desafios técnicos enfrentados em Prompt Engineering, a arquitetura com Llama 3.3 70B e a aderência integral a todos os requisitos.

6. Conclusão

Ao longo das quatro semanas de desenvolvimento, o projeto DataChat SQL Lite evoluiu de um estudo inicial do domínio de dados Olist até uma aplicação funcional e estável, capaz de traduzir perguntas em linguagem natural para consultas SQL executadas sobre um banco DuckDB. A arquitetura modular — separando camada de dados, orquestração LLM e interface — permitiu tratar erros de forma resiliente, validar consultas complexas com múltiplas tabelas e entregar explicações em linguagem natural para o usuário final.

Como resultado, todos os requisitos funcionais planejados (incluindo resiliência a erros, suporte a consultas com JOIN e histórico de interações) foram validados com sucesso, consolidando o DataChat SQL Lite como uma prova de conceito robusta para interfaces conversacionais aplicadas à análise de dados de e-commerce.